

Regularz - Team16

Regularz - Team16 (*student proposal*)

Assessment: ♦ Fit with adjustments · **Category:** GovTech · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): A mobile pickup-ordering app giving small Singapore F&B businesses a direct brand channel (vs commission-heavy marketplaces like Grab/Foodpanda or generic POS terminals). Customers browse meals/drinks with live stock, order made-to-order food, and get collection ETAs from a C++ queue-estimation service (moving-window / rolling-average of per-item prep time against the active-order backlog). Collection-only: a UUID order-bound QR code is generated after payment and scanned on the vendor device to confirm pickup. Inventory updates concurrently and safely (DB transactions reserve/restore stock, preventing overselling the last item). A data-driven CRM uses purchase histories to deliver personalized rewards (not flat points). Load-bearing C++: a cloud-hosted queue-management service on Linux that manages the live order queue, verifies payment/kitchen-status updates, computes ETAs, and stores order/ETA data in Firebase with transaction-safe atomic processing, pushing real-time updates to customer phones and the vendor tablet (mobile web). Custom tool: an F&B Operations Editor (staff preview/modify stock, add/remove menu items, edit loyalty rewards, customize the interface). CI/deployment flow with staging→production and regression gates. Design features: branded mobile menu, end-to-end pickup ordering with live collection tracking + QR, vendor dashboard (orders/inventory), and personalized loyalty (a virtual wallet tracking purchases/rewards).

Category note: filed under **GovTech** — the load-bearing spine mirrors GovTech's "queue management" example (simulate/track a queue, real-time dashboard, service-flow/ops editor) as a **two-role service platform** (customer + vendor, order → prepare → ready → collected workflow with status tracking). Runner-ups: **CMS** (its "restaurant menu & ordering" example — menu authoring + customer ordering) and **Fintech** (payment/wallet/transaction-safe), which fit narrower slices.

Adjustments required for approval:

- **Clear the 2D gate with a custom-rendered data surface.** The vendor dashboard (live order queue, inventory, ETA/collection tracking) should be a genuinely **custom-rendered OpenGL ES data-display surface** (like the GovTech dashboard-rendering goal), not only native widgets/tables — otherwise the app risks collapsing into ordinary CRUD.
- **Payments are mock/sandbox — no real money.** Use a simulated/sandbox payment step (no live payment gateway, no real funds), and treat the "virtual wallet" as **in-app loyalty points**, not real currency. This keeps the project in scope for the module (and out of Fintech's regulated territory).
- **Keep the queue/ETA + concurrency-safe transactions as the team's own load-bearing C++** (which the proposal commits to). The **QR order-collection pipeline** and the **personalized CRM/loyalty engine** are novel for this category's civic-flavoured pool — propose them as instructor-approved new pool goals (see below).

References:

- [Toast \(toasttab.com\)](https://toasttab.com) — [Wikipedia: a cloud restaurant POS + mobile ordering + customer loyalty/promotions platform \(a direct-brand alternative to marketplaces and dedicated terminals\)](#)
- [Ritual \(ritual.co\)](https://ritual.co) — an order-ahead / skip-the-line pickup app that positions against commission-heavy marketplaces and gives restaurants an affordable direct digital channel
- [Olo \(olo.com\)](https://olo.com) — a restaurant technology platform for direct branded digital ordering, payments and loyalty (not a marketplace)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✗ Not needed | » Second trimester

Core Tech Requirement	Regularz
External Database	✓
Authentication »	✓
Analytics »	✓
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	♦
3D (strong students only)	✗
Networking System	✓

How each requirement applies:

Platform fit: **Main:** Web browser on mobile — the customer app and the vendor tablet both run as a mobile web front end (HTML/JS/CSS + the C++ engine compiled to WASM via Emscripten), touch-first (device: Pixel 9a); the C++ queue service runs on a Linux cloud host. **Extended:** Firebase + the Linux server (already the home of the queue service) set up by M2 (full use T2).

Front end: The branded menu with live stock, item customization + order flow, live collection/ETA tracking and the QR screen (customer); and the vendor dashboard (order queue, inventory, ready/collected) — with a custom-

rendered queue/ETA/inventory data surface (OpenGL ES / WebGL).

Languages: Web — HTML/JS/CSS + C++→WASM (Emscripten, SDL2/3, OpenGL ES/WebGL). (Web-on-mobile target — no native app build this trimester; the C++ service is Linux-hosted.)

Back end: The cloud-hosted C++ queue-management service — live order queue, payment/kitchen-status verification, rolling-average ETA computation, concurrency-safe stock reserve/restore, and the CRM/loyalty engine — with Firebase for orders/records/roles and real-time sync. **Languages:** C++.

Backend scope: Server-indicated (T2) — the Linux-hosted queue service is set up & basic-tested by M2, full use in T2; Firebase is the T1 store/sync bridge.

- *Loop and/or service:* the load-bearing piece is a **performant C++ service** — it maintains the live order queue, computes each order's collection ETA from the active backlog and a rolling average of per-item prep/collection time, verifies payment/kitchen-status updates, and reserves/restores stock with **transaction-safe atomic operations** (no double-selling the last item, correct restock on cancel) even under duplicate/interrupted requests.
- *Data-driven:* the menu/items, stock, prep-time parameters, order states, loyalty/reward rules and the ops-editor interface config are hot-reloadable JSON — the F&B ops editor tunes them without recompiling.
- *2D:* the vendor dashboard (order queue, inventory, ETA/collection tracking) and the customer live-tracking view are **custom-rendered data-display UI** in OpenGL ES (see adjustment 1). *3D:* not needed. *Networking:* real-time push of order/ETA/status updates to customer phones and the vendor tablet.
- *Custom tool:* the **F&B Operations Editor** — staff preview/modify stock, add/remove menu items, edit loyalty rewards, and customize the interface (a form/flow-builder-style editor).
- *External DB:* Firebase stores orders/records/roles and syncs in real time. *Auth »:* customer + vendor accounts and roles. *Analytics »:* purchase histories drive the CRM/loyalty.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the GovTech pool in `milestone-goals-pool-per-category.pdf` (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** C++17/20 with a fixed-timestep service loop driving the queue tick, ETA recompute and status updates; SDL2/3 dev harness, CMake, Git; Emscripten→WASM for the mobile-web front end; the service also runs headless on the Linux host.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the M1G4 queue engine and M1G6 order workflow run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** C++ entity/registry with menu items, stock, prep-time parameters, orders + states, loyalty/reward rules and the ops-editor config in JSON (nlohmann/json) with hot-reload.
- **Prerequisites:** G1 (loop & update hooks).
- **Feeds into:** M1G4 (queue reads the model), M1G6 (order states), M2G2 (the ops editor edits it), M2G10 (ETA features).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays for the queue/ETA and order states, Tracy profiling, Catch2 tests for the transaction-safe stock logic (concurrent-order race: no oversell) and ETA determinism, GitHub Actions Windows+Linux matrix + the staging→production regression gates the proposal describes.
- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening) and the ETA/transaction regression suite.

- **G4 Real-time Feed / Queue Engine** — the live order queue: track paid orders + kitchen availability and derive collection ETAs from the backlog.
Tech & dependencies
 - **Tech:** A C++ queue engine ticking active orders, a rolling average of per-item prep time, and backlog-based ETA updates emitted on order create/complete.
 - **Prerequisites:** G1 (loop), G2 (order/item model).
 - **Feeds into:** M2G10 (ETA prediction matured); the customer live-tracking view and vendor dashboard.
- **G6 Workflow / State Machine Engine** — the order lifecycle (created → paid → preparing → ready → collected / cancelled) with concurrency-safe stock reserve/restore.
Tech & dependencies
 - **Tech:** A C++ state machine over order statuses with transaction-safe transitions that reserve stock on order and restore it on eligible cancel, guarding against duplicate/interrupted requests (atomic operations).
 - **Prerequisites:** G2 (order/stock model).
 - **Feeds into:** The QR-collection step; M2G7 (audit trail of transitions).
- **G9 Staff / Officer Dashboard — Foundation** — the vendor dashboard: order queue, inventory and ready/collected management.
Tech & dependencies
 - **Tech:** A vendor-side dashboard (custom-rendered) showing the live order queue, per-item stock and order actions (mark ready, confirm collected); reads the queue engine + order states.
 - **Prerequisites:** M1G4 (queue), M1G6 (order states), 2D rendering.
 - **Feeds into:** M2G4 (advanced dashboard), the QR-scan collection flow.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**
Tech & dependencies
 - **Tech: Web build:** Tracy profiling of the queue/ETA/transaction paths, ASan on MSVC, and Emscripten→WASM validation on the mobile-web target (customer phone + vendor tablet) on the Pixel 9a; the Linux queue service set up (account + software + basic connectivity test) by M2.
 - **Prerequisites:** All M1 goals (hardens the M1 codebase).

- **Feeds into:** The mobile-web deliverable; the Linux-hosted service for T2.
- **G2 ★ Content Editor — Core** — the F&B Operations Editor (form/flow builder): edit stock, add/remove menu items, edit loyalty rewards, and customize the interface.
Tech & dependencies
 - **Tech:** C++ + Dear ImGui (or a web front-end), JSON serialization, undo/redo; live preview so a stock/menu/reward or layout change reflects immediately in the app.
 - **Prerequisites:** M1G2 (data-driven architecture).
 - **Feeds into:** M2G6 (advanced builder features); the vendor's day-to-day operations.
- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**
Tech & dependencies
 - **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, packaging of the Emscripten web build + the Linux service, and the staging→production promotion flow with ETA/transaction regression gates via GitHub Actions.
 - **Prerequisites:** G3 (CI and test foundation).
 - **Feeds into:** Reliable deploys of the queue service + mobile web app.
- **G10 Prediction Engine** — collection-time ETA matured (wait-time / ready-time estimates from the backlog).
Tech & dependencies
 - **Tech:** A refined estimator over the rolling average of per-item prep + collection time and the active backlog (peak-period aware), with accuracy evaluation against recorded actuals.
 - **Prerequisites:** M1G4 (queue engine + prep-time data).
 - **Feeds into:** The customer ETA and the peak-period time-management the menu surfaces.
- **G8 Firebase — Auth Roles, Sync & Analytics** — customer + vendor accounts/roles, real-time order sync, and purchase-history analytics for the CRM.
Tech & dependencies
 - **Tech:** Firebase Auth roles (customer/vendor), Firestore/Realtime DB for orders/records with real-time sync to phones + the vendor tablet, and Analytics feeding the loyalty/CRM.
 - **Prerequisites:** M1G2 (data model to sync).

- **Feeds into:** The CRM/loyalty engine; role-gated vendor actions.
- **G7 RBAC & Audit Logging** — staff roles/permissions + a decision/transaction trail for orders and payments.

Tech & dependencies

- **Tech:** Role-based access control (owner/staff), and an append-only audit log of order transitions, stock changes and (mock) payment events — supporting the transaction-safe guarantees.
- **Prerequisites:** M1G6 (order transitions); Firebase identity.
- **Feeds into:** Trust/accountability; dispute resolution on collection.

(Proposed new pool goals (require instructor approval): **QR Order-Collection**

Pipeline — a UUID order-bound code becomes a unique QR after (mock) payment, scanned/decoded on the vendor device to mark the order collected; and **Personalized CRM & Loyalty Engine** — purchase-history-driven rewards and an in-app points wallet (not flat points). Neither exists in the GovTech pool; the three picks above stand regardless.)

Track Goals & Team Composition

Recommended composition: 6 CS + 2 Design + 0 Art (8) — this is a two-role service platform (customer + vendor) with heavy service-flow and dashboard/ops UX and minimal illustrated art (menu photos are vendor-supplied; the rest is UI/branding), so two designers and no dedicated artist (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). With 0 artists, the designers take the **D11 conditional** as a real graded Design goal each milestone (M1 Art Proof of Concept / M2 Art Implementation Quality) — the last Design goal each milestone.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (customer + vendor blueprint; the browse → order → pay → track → collect loop + the loyalty model)
- M1D2 Prototype (both roles clickable) · prototype in Figma
- M1D6 Service Journey Design (order → paid → preparing → ready → collected, incl. cancel / no-show / out-of-stock paths)
- M1D3 User Research, Personas & Competitive Analysis (small F&B owners + regulars; Grab/Foodpanda/EPOS/Ritual teardown)
- M1D11 Art POC (conditional — no artist) — the design-owned art surface (style guide, iconography, UI, menu-card system)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D6 Service polished end to end (order → ETA → QR → collected, both roles)
- M2D8 Status & Notification pass (order-ready, ETA changes, collection confirm — reassurance at every state)
- M2D11 Art Implementation Quality (conditional — no artist) — implemented art aligned with the POC, no placeholders

- **UI/UX M1** (graded by the UI/UX instructor):

- M1U1 ★ UI Standards & Wireframe Baseline — customer + vendor screen-flow map and readability standards.

- M1U2 UX Flows & Wireframes — browse → order → track → collect; vendor order-management.
- M1U3 Accessibility & Readability Foundations — legible menu/queue typography, contrast, thumb-reach targets.

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — customer AND vendor roles navigable unaided on the mobile-web target.
- M2U2 ★ UX Validation & Final Polished UI — validated menu, tracking and vendor dashboard.
- M2U3 Menu & HUD Motion Design — order-state transitions, ETA updates, QR reveal.
- M2U4 Onboarding & Tutorial Design — vendor setup + first-order guidance.

- **Art:** 0 artists — the art surface is covered by the M1D11/M2D11 conditionals, taken as Design goals (see Design M1/M2).

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial UI/notification cues from the week-5 needs list present & working in the prototype (order placed, paid, ready, collected, error), suitable and correctly triggered.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — GovTech floor (no BGM; ≥5 restrained cues, each paired with a visual state change — placed / paid / ready / collected / error).
 - M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel; each cue distinct on the vendor tablet vs the customer phone).
 - M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the F&B Operations Editor), prototype demo, team post-mortem (wk14)
-

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*